POINTERS

Introduction

- Pointer is a derived data type in C.
- Pointers contain memory addresses as their values.
- Pointers can be used to access and manipulate the data stored in memory.

Understanding Pointers

- Pointer is a variable that can hold the address of another variable or address of a memory location.
- Like any other variable or a constant we should declare a pointer variable before using it.

Benefits of using pointers in C

- Pointers provide direct access to memory.
- Pointers are used to access the value of the variable through its address.
- Pointers provide a way to return more than one value to functions.

Benefits of using pointers in C(contd..)

- Addresses of the objects can be extracted using pointers.
- Reduces the storage space and complexity of the program.
- Reduces the execution time of the program.

Accessing the address of a variable

- To access the address of a variable **&** (ampersand) operator is used.
- Let us declare and initialize two variables ‘a’ and ‘b’.

int a=100; int b=450;

- Let the starting address of variable a is 1000 and variable b is 1002.
- Address of variable a is accessed using &a and address of variable b is accessed as &b.

Program to print the value and address of two variables:

```c
#include <stdio.h>
int main()
{
        int a=100;
        int b=450;
        printf("The value of variable a=%d\n", a);
        printf("The value of variable b=%d\n", b);
        printf("Address of a= %d and b=%d", &a, &b);
        return 0;
}
```

Output of the program

The value of variable a=100

The value of variable b=450

Address of a=1000 and b=1002

Declaring Pointer variables

- General form of declaration:

type *variablename;

Here type is the pointer base type and must be valid data type.

- Asterisk(*) is used to declare pointer variable.

Declaration and initialization of Pointer Variable

A pointer is declared by specifying its name and type, just like simple variable declaration but with an asterisk (*) symbol added before the pointer's name.

- data_type* name int *p;
- Pointer initialization means assigning some address to the pointer variable.

```
int var = 10;
// Initializing ptr
 int *ptr = &var;
```

Some of the valid pointer declarations are:

1) **int *p;**

This declaration means p is a pointer to an integer.

2)**char *ch;**

This declaration means ch is a pointer to a character.

Steps to be followed to use pointers

- Declare a data variable

  int x;

- Declare a pointer variable

  int *p;

- Initialize pointer variable

  p=&x;

- Access data using pointer variable

Dereference a Pointer

- We have to first dereference the pointer to access the value present at the memory address. This is done with the help of dereferencing operator(*)

```
int main() {
    int var = 10;
    // Store address of var variable
    int* ptr = &var;
    // Dereferencing ptr to access the value
    printf("%d", *ptr);
    return 0;
}
```

Simple program on pointers

Programming example:

This program declares and initializes a variable and prints the address of variable and illustrates the important operations done with the help of pointers, that is :

(a) We define a pointer variable,

(b) assign the address of a variable to a pointer and

(c) finally access the value at the address available in the pointer variable.

Program 1:

```
#include<stdioh>
int main(){
    int var=20;
    int *ip;
    ip=&var;
    printf("address of var =%x\n",&var);
    printf("address stored in ip =%x\n",ip);    printf("Value of
    *ip variable=%d", *ip); /*accessing the value using pointer*/
    return 0;
```

Output of program 1

address of var =bffd8b3c

address stored in ip= bffd8b3c

Value of *ip variable=20

Simple program on pointers

Program 2:

Write a C program to add two numbers using pointers

Program 2:To add two numbers

```
#include <stdio.h>

int main ()

{

    int  a=10,b=20,sum;

    int *p1, *p2;

    p1=&a;

    p2=&b;

    sum=*p1+*p2;

    printf(“\n Sum=%d” sum); return 0;
```

Output program2

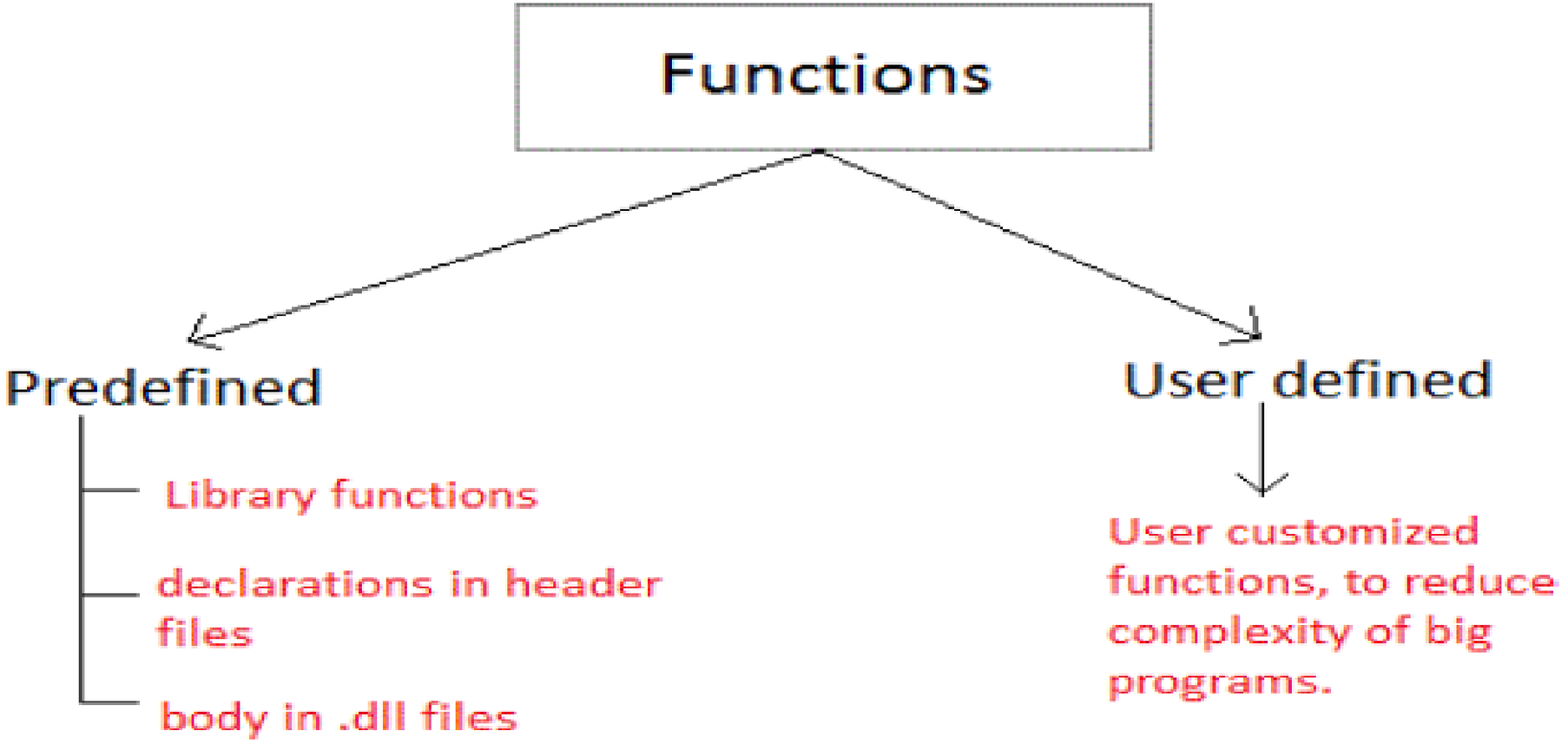
Sum=30

User-defined Functions

- There are many situations where we might need to write same line of code for more than once in a program.
- This may lead to unnecessary repetition of code, bugs and even becomes boring for the programmer.
- So, C language provides an approach in which you can declare and define a group of statements once in the form of a function and it can be called and used whenever required.
- These functions defined by the user are also know as User-defined Functions.

- C functions can be classified into two categories,

1. Library functions
2. User-defined functions



BENEFITS OR NEED OF USING FUNCTIONS

- It provides modularity to program's structure.
- It makes your code reusable.
- Debugging and editing becomes easier.
- Program is readable and easy to understand.
- Functions save both time and space.
- A function may be used by many other programs.

ELEMENTS OF USER-DEFINED FUNCTIONS

- There are three elements related to functions

1. Function declaration
2. Function definition
3. Function call

Function Declaration

- General syntax for function declaration,
- **Returntype functionName(type1 parameter1, type2 parameter2,...);**
- Function declaration informs the compiler about the function name, parameters it accepts, and its return type.
- It's also called as **Function Prototyping**.

- Function declaration consists of 3 parts.
- **returntype**
- Return type specifies the type of value(int, float, char, double) that function is expected to return to the program which called the function.
- **Note: In case your function doesn't return any value, the return type would be void.**
- **functionName**
- Function name is an identifier and it specifies the name of the function.
- **parameter list**
- The parameter list declares the type and number of arguments that the function expects when it is called.

Function definition

- The function definition is an independent program module that is specially written to implement the requirements of the function.
- The general syntax of function definition is,

```
returntype functionName(type1 parameter1, type2 parameter2,...)
{
// function body goes here
}
```

- The first line returntype **functionName(type1 parameter1, type2 parameter2,...) is known as function header and the statement(s) within curly braces is called function body.**

- The function body is enclosed within curly braces { ... } and consists of three parts.
- **local variable declaration(if required).**
- **function statements to perform the task inside the function.**
- **a return statement to return the result evaluated by the function(if return type is void, then no return statement is required).**

- **Calling a function:**
- When a function is called, control of the program gets transferred to the function. functionName(argument1, argument2,...);
- **PASSING ARGUMENTS TO A FUNCTION**
- Arguments are the values specified during the function call.

Functions

With Arguments

- declared and defined with parameter list
- values for parameter passed during call
- Eg :

```
// declaration
int sum (int x, int y);
//call
sum(10, 20);
```

Without Arguments

- No parameters included.
- No value passed during function call
- Eg :

```
// declaration
int display();
// call
display();
```

```c
#include<stdio.h>

int multiply(int a, int b);

int main()
{
    ... ...
    result = multiply(i, j);
    ... ...
}

int multiply(int a, int b)
{
    ... ...
}
```

providing arguments while calling function

Returning a value from function:

- Return statement also ends the function execution, hence it must be the last statement of any function

```
#include<stdio.h>

int multiply(int a, int b);

int main()
{
    ... ...
    result = multiply(i, j);
    ... ...
}

int multiply(int a, int b)
{
    ... ...
    return a*b;
}
```

The value returned by the function must be stored in a variable.

- A function may have more than one return statements.

```
fun()
{
if(x <= 0)
    return(0);
else
    return(1);
}
```

Actual Parameters and Formal Parameters in C

- Actual parameters are the values that are passed to the function during a function call. They are also known as arguments.
- Formal parameters are also called function parameters or function arguments. They are used in the function definition to accept the values from the caller of the function.

```c
#include<stdio.h>
    int add(int a, int b){
    int sum = a + b;
    return sum;
}
int main()
{
    int sum = add(10, 20);
    return 0;
}
```

In the actual and formal parameters example, **a and b are formal parameters**. When this function is called with actual parameters, these parameters are passed to the function as arguments. Here, 10 and 20 are actual parameters that are passed to the add() function.

Actual Parameters	Formal Parameters
1. Actual parameters are the values or references that are passed into the function when it is called.	1. They can have default values, which are used if no value is passed in as an actual parameter when the function is called.
2. They are also called arguments or parameter values.	2. Formal parameters are also called function parameters or function arguments.
3. Actual parameters provide input to the function.	3. They are the placeholders for the values or references that will be passed into the function.
4. They have a scope that is limited to the function call in which they are used.	4. Formal parameters have a scope that is limited to the function in which they are defined.
5. Actual parameters can be expressions that are evaluated before they are passed into the function.	5.They are used by the function to perform operations and computations.

Scope of variables

- The scope of a variable in C is the block or the region in the program where a variable is declared, defined, and used.
- Outside this region, we cannot access the variable and it is treated as an undeclared identifier.
- The scope is the area under which a variable is visible. We can only refer to a variable in its scope.

Local Scope of Variables in C

```
#include <stdio.h>
int main() {
    int my_num = 7;
    printf("%d",my_num);
    my_func();
    return 0;
}
void my_func() {
   printf("%d",my_num);
}
```

- error: 'my_num' undeclared (first use in this function)

Global Scope of Variables in C

```
#include <stdio.h>
int my_num = 7;
int main() {
    printf("my_num can be accessed from main() and its value is %d\n",my_num);
    //call my_func my_func();
    return 0;
}
void my_func() {
    printf("my_num can be accessed from my_func() as well and its value is
      %d\n",my_num);
}
```

- //Output
- my_num can be accessed from main() and its value is 7 my_num can be accessed from my_func() as well and its value is 7

CATEGORIES OF USER-DEFINED FUNCTIONS IN C

Function with no arguments and no return value:

```
#include<stdio.h>
void greatNum(); // function declaration
int main()
{
	greatNum(); // function call return 0;
}
void greatNum() // function definition
{
	int i, j;
	printf("Enter 2 numbers that you want to compare...");
	scanf("%d%d", &i, &j);
	if(i > j) {
		printf("The greater number is: %d", i);
	}
	else
	{
		printf("The greater number is: %d", j);
	}
}
```

Function with no arguments and a return value:

```
#include<stdio.h>
int greatNum(); // function declaration
int main()
{
        int result;
        result = greatNum();
        // function call
        printf("The greater number is: %d", result);
        return 0;
}
int greatNum() // function definition
{
        int i, j, greaterNum;
        printf("Enter 2 numbers that you want to compare...");
        scanf("%d%d", &i, &j);
        if(i > j)
        {
                greaterNum = i;
        }
        else
         {
                greaterNum = j;
        }
```

Function with arguments and no return value:

```
#include<stdio.h>
void greatNum(int a, int b); // function declaration
int main()
{
	int i, j;
	printf("Enter 2 numbers that you want to compare...");
	scanf("%d%d", &i, &j);
	greatNum(i, j); // function call
	return 0;
}
void greatNum(int x, int y) // function definition
{
	if(x > y) {
	printf("The greater number is: %d", x);
	}
	else {
	printf("The greater number is: %d", y);
	}
}
```

Function with arguments and a return value:

```
#include<stdio.h>
int greatNum(int a, int b); // function declaration
int main()
{
	int i, j, result;
	printf("Enter 2 numbers that you want to compare...");
	scanf("%d%d", &i, &j);
	result = greatNum(i, j); // function call
	printf("The greater number is: %d", result);
	return 0;
}
int greatNum(int x, int y) // function definition
{
	if(x > y)
	{
		return x;
	}
	else
	{
		return y;
	}
```

C program to add two integers

```
#include <stdio.h>
int addNumbers(int a, int b); // function prototype
int main() {
        int n1,n2,sum;
        printf("Enters two numbers: ");
        scanf("%d %d",&n1,&n2);
        sum = addNumbers(n1, n2); // function call
        printf("sum = %d",sum); return 0;
}
int addNumbers(int a, int b) // function definition {
        int result;
        result = a+b;
        return result; // return statement
}
```

C program to find the maximum of two integers

```
#include <stdio.h>
int max(int num1, int num2) {
   if (num1 > num2)
      return num1;
   else
      return num2;
}
int main() {
   int a = 10, b = 20, c;
   c = max(a, b);
printf("The maximum of %d and %d is %d\n", a, b, c);
   return 0;
}
```

C program to find factorial of a number

```
#include <stdio.h>
int fact(int);
void main()
{
    int no,factorial;
    printf("Enter a number: \n");
    scanf("%d",&no);
    factorial=fact(no);
    printf("The Factorial of %d is %d\n",no,factorial);
}

int fact(int n)
{
   int i,f=1;
   for(i=1;i<=n;i++)
   {
      f=f*i;
   }
   return f;
}
```

```c
/* * C program to find the GCD of Two Numbers using Euclidean Algorithm */
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
int gcd(int x, int y) {
        int r = 0, a, b;
        a = (x > y) ? x : y; // a is greater number
        b = (x < y) ? x : y; // b is smaller number
        r = b;
        while (a % b != 0) {
                r = a % b;
                a = b;
                b = r;
        }
        return r;
}
int main() {
        printf("Enter the two numbers: \n");
        int x, y; scanf("%d", &x);
        scanf("%d", &y);
        printf("The GCD of two numbers is: %d", gcd(x, y));
```

Call By Value and Call By Reference

Call By Value

- In **call by value**, a copy of the variable's value is passed to the function.

- Changes made inside the function **do not** affect the original variable.

Example Program 1:

```c
#include <stdio.h>

void changeValue(int x) {
    x = 100; // changes only the copy
}

int main() {
    int a = 10;
    changeValue(a);
    printf("Value of a: %d\n", a); // a is still 10
    return 0;
}
```

Output

```
Value of a: 10
```

Example Program 2:

```c
#include <stdio.h>

// Function that takes value as parameter
void swapValue(int x, int y) {
    int temp = x;
    x = y;
    y = temp;
    printf("Inside swapValue: x = %d, y = %d\n", x, y);
}

int main() {
    int a = 5, b = 10;
    printf("Before swapValue: a = %d, b = %d\n", a, b);
    swapValue(a, b);
    printf("After swapValue: a = %d, b = %d\n", a, b); // No change
    return 0;
}
```

Output:

```
Output
Before swapValue: a = 5, b = 10
Inside swapValue: x = 10, y = 5
After swapValue: a = 5, b = 10
```

Call By Reference

- In **call by reference**, the function receives the address of the variable.

- Changes made inside the function **do** affect the original variable.

Example Program 1:

```c
#include <stdio.h>

void changeValue(int *x) {
    *x = 100; // changes the actual variable
}

int main() {
    int a = 10;
    changeValue(&a);
    printf("Value of a: %d\n", a); // a is now 100
    return 0;
}
```

Output

```
Value of a: 100
```

Example Program 2:

```c
#include <stdio.h>

// Function that takes addresses as parameters
void swapReference(int *x, int *y) {
    int temp = *x;
    *x = *y;
    *y = temp;
    printf("Inside swapReference: x = %d, y = %d\n", *x, *y);
}

int main() {
    int a = 5, b = 10;
    printf("Before swapReference: a = %d, b = %d\n", a, b);
    swapReference(&a, &b);
    printf("After swapReference: a = %d, b = %d\n", a, b); // Values
        swapped
    return 0;
}
```

Output:

```
Output

Before swapReference: a = 5, b = 10
Inside swapReference: x = 10, y = 5
After swapReference: a = 10, b = 5
```

Storage Classes in C:

What are Storage Classes?

In C, storage classes define:

- **Scope** – Where the variable is accessible (inside function, across files, etc.)
- **Lifetime** – How long the variable exists in memory
- **Default value** – What it’s initialized to if not assigned

C has **four** main storage classes:

- **auto**
- **register**
- **static**
- **extern**

1. auto (Default for local variables)

- **Scope:** Local to the block/function
- **Lifetime:** Created when the function is called, destroyed when it ends
- **Default value:** Garbage value

```c
#include <stdio.h>

void demoAuto() {
    auto int x = 5; // 'auto' is optional here
    printf("Auto variable: %d\n", x);
}

int main() {
    demoAuto();
    return 0;
}
```

Output

```
Auto variable: 5
```

2. register

- **Scope:** Local to the block/function
- **Lifetime:** Same as auto, but stored in CPU register (if possible) for faster access
- **Default value:** Garbage value
- **Note:** You **cannot** take the address of a register variable.

```c
#include <stdio.h>

void demoRegister() {
    register int count = 10;
    printf("Register variable: %d\n", count);
}

int main() {
    demoRegister();
    return 0;
}
```

Output

```
Register variable: 10
```

3. static

- **Scope:** Local to function (if declared inside function) or global (if outside function)
- **Lifetime:** Exists for the entire program duration
- **Default value:** 0 if not initialized
- **Special:** Retains value between function calls

```c
#include <stdio.h>

void demoStatic() {
    static int counter = 0;
    counter++;
    printf("Static counter: %d\n", counter);
}

int main() {
    demoStatic();
    demoStatic();
    demoStatic();
    return 0;
}
```

Output

```
Static counter: 1
Static counter: 2
Static counter: 3
```

4. extern

- **Scope:** Global across multiple files
- **Lifetime:** Exists for the entire program
- **Default value:** 0
- **Usage:** Declares a variable defined in another file or later in the same file

```c
#include <stdio.h>

int num = 100; // Global variable

void showExtern() {
    extern int num; // Refers to global variable
    printf("Extern variable: %d\n", num);
}

int main() {
    showExtern();
    return 0;
}
```

Output

```
Extern variable: 100
```

Recursive Function

- A **recursive function** is a function that **calls itself** either directly or indirectly to solve a problem.
- It is often used for problems that can be broken into smaller, similar sub-problems (e.g., factorial, Fibonacci series)

- Every recursive function must have a **base case** to stop recursion. Without a base case, the recursion will cause **infinite calls.**

Factorial of a Number using Recursion

```c
#include <stdio.h>

// Function to calculate factorial recursively
int factorial(int n) {
    if (n == 0 || n == 1)  // Base case
        return 1;
    else
        return n * factorial(n - 1);  // Recursive call
}

int main() {
    int num = 5;
    printf("Factorial of %d is %d\n", num, factorial(num));
    return 0;
}
```

Output

```
Factorial of 5 is 120
```

Fibonacci Series using Recursion

```c
#include <stdio.h>

// Function to return nth Fibonacci number
int fibonacci(int n) {
    if (n == 0)  // Base case 1
        return 0;
    else if (n == 1)  // Base case 2
        return 1;
    else
        return fibonacci(n - 1) + fibonacci(n - 2);  // Recursive
            calls
}

int main() {
    int i, terms = 6;
    printf("Fibonacci series: ");
    for (i = 0; i < terms; i++) {
        printf("%d ", fibonacci(i));
    }
    return 0;
}
```

Output

```
Fibonacci series: 0 1 1 2 3 5
```